

JUG Stuttgart · Demo-Recap

Lokale LLMs auf gemieteten GPUs

Ein Befehl. Eine GPU für eine Stunde. Dein eigenes Modell.

Die einfache Variante — ohne Browser-IDE.

30. April 2026 · Uni Stuttgart, Raum 1.311

Tool: gpu-summon · github.com/DG1001/gpu-summon

Im Kontext des Agentic-Coding-Workshops, aber unabhängig von den AGENTS.md-Übungen

Was du auf der Bühne gesehen hast

In drei Schritten — und alles war echt live

1. Ein Befehl

```
python summon.py --solo-mode
```

Suche nach passender GPU, Anmieten, Container hochfahren, llama-server starten — alles in einem Aufruf.

2. ~3-5 Min warten

```
Modell wird einmalig geladen
```

Das GGUF (~21 GB) zieht von Hugging-Face direkt auf die GPU. Danach lebt der Endpoint.

3. Lokal: opencode

```
Konfiguration ist schon da
```

opencode-Provider und Bearer-Token sind automatisch eingetragen. Einfach opencode starten — fertig.

Wenn fertig: ein Kommando räumt alles auf.

```
python summon.py --destroy <id> → Container weg, Stundenzähler aus.
```

Wofür man so etwas braucht

Die drei Gründe, die immer wieder genannt werden



Daten bleiben deins

Code geht an DEINEN Server.
Kein Cloud-LLM-Anbieter sieht ihn.
Wichtig für NDA-Projekte, Beratung,
Forschung.



Bezahlbar nach Stunde

~30-40 ¢/h auf RTX 5090.
8 h Workshop-Tag ≈ 3 \$.
Keine Abo-Falle, keine Token-Pakete.



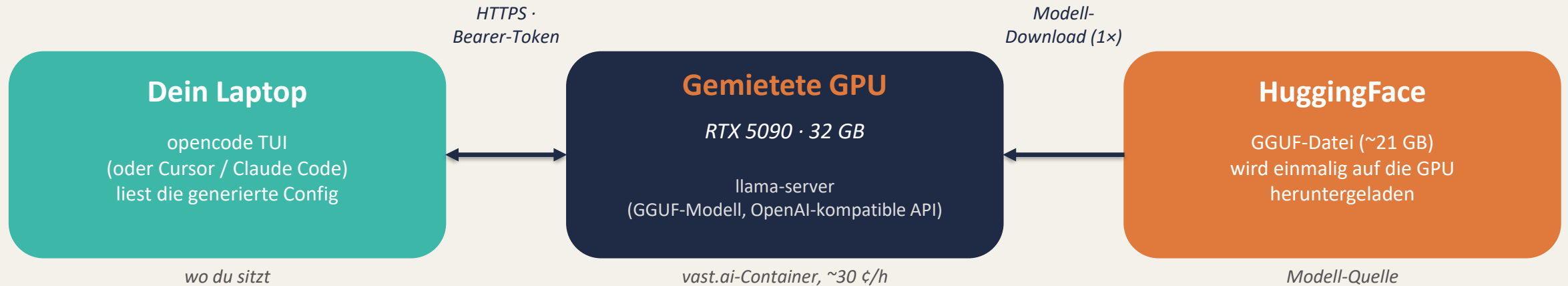
Volle Modell-Kontrolle

Du wählst Modell + Quantisierung +
Context-Größe selbst.
Heute Qwen, morgen DeepSeek,
übermorgen Llama.

Die Schwelle, ab der sich's lohnt: lokale GPU zu klein, Cloud-API tabu oder zu teuer.

Die Idee in einem Bild

Drei Stationen — und in der Mitte lebt deine GPU für eine Stunde



Wichtig: dein Code geht NUR an die gemietete GPU.

Kein Anthropic, kein OpenAI, kein Mistral sieht den Inhalt deiner Files.

Was passiert hinter dem einen Befehl?

Die fünf Schritte, die `summon.py` für dich erledigt

```
$ python summon.py --solo-mode
```

1. Suchen

Filter:
VRAM, Preis,
Reliability,
Bandbreite

2. Mieten

Beste Offer
starten —
Container
fährt hoch

3. Modell laden

GGUF von
HuggingFace
auf die GPU
(~21 GB)

4. Server

llama-server
startet,
/v1/chat
live

5. Config

opencode-
Provider +
Bearer-Token
schreiben

Was du am Ende auf der Konsole siehst:

- Endpoint: **`https://78.55.x.x:46291/v1`**
- Bearer: `ZeFi0s9k...` (32 chars, auto-generated)
- Config: `~/.config/opencode/opencode.json` (geschrieben)
- Auth: `~/.local/share/opencode/auth.json` (geschrieben)

Damit du es selber probieren kannst

Voraussetzungen und der Mini-Quickstart

Du brauchst

- ✓ **Python ≥ 3.10**
auf deinem Laptop (Linux/macOS/WSL)
- ✓ **vast.ai-Account**
kostenlos · API-Key unter Manage Keys
- ✓ **\$5–10 Guthaben**
auf vast.ai, deckt viele Coding-Sessions
- ✓ **opencode**
lokal · der Launcher schreibt seine Config direkt mit
- ✓ **ssh + ssh-keygen**
Standard auf Linux/macOS · nur für Smoketest nötig

```
quickstart - bash

# Repo holen
git clone https://github.com/
  DG1001/gpu-summon.git
cd gpu-summon

# Abhängigkeiten
pip install vastai requests

# API-Key setzen (einmalig)
export VAST_API_KEY=...

# Los – Solo-Mode, secure-by-default
python summon.py --solo-mode

# Wenn fertig
python summon.py --list
python summon.py --destroy <id>
```

Repo: github.com/DG1001/gpu-summon · README hat Schritt-für-Schritt für vast.ai-Setup

Drei Betriebs-Modi

Damit du nicht jedes Mal Slot- und Context-Math machst

Modus	Befehl	Slots	Total Context	Pro Slot	Wann?
default	(ohne Flag)	4	65 k	16 k	kleines Team, mittlere Chats
workshop	--workshop-mode	8	131 k	16 k	5–10 User parallel — Demo, Workshop
solo	--solo-mode	2	131 k	65 k	ein User, lange Sessions, große Repos

Eigene Werte? Override mit:

```
python summon.py --parallel 6 --num-ctx 96000
```

Die Decke auf 32-GB-Karten liegt bei ~256 k Total-Context (mit q8 KV-Cache).

Sicherheit ab Werk

Warum dein Endpoint nicht offen im Netz steht

Ohne Auth (DON'T)

Endpoint ist über Random-IP + Random-Port erreichbar — aber Bots scannen vast.ai-Subnets aktiv. In Stunden steht ein fremder Workload auf DEINER Stundenrechnung.

Default: Bearer-Token

32-Char-Token wird beim Launch generiert und an llama-server als --api-key gegeben. Ohne Header → 401. opencode kriegt den Token automatisch in die auth.json geschrieben.

```
# Default – auto-generiert
python summon.py \
  --solo-mode

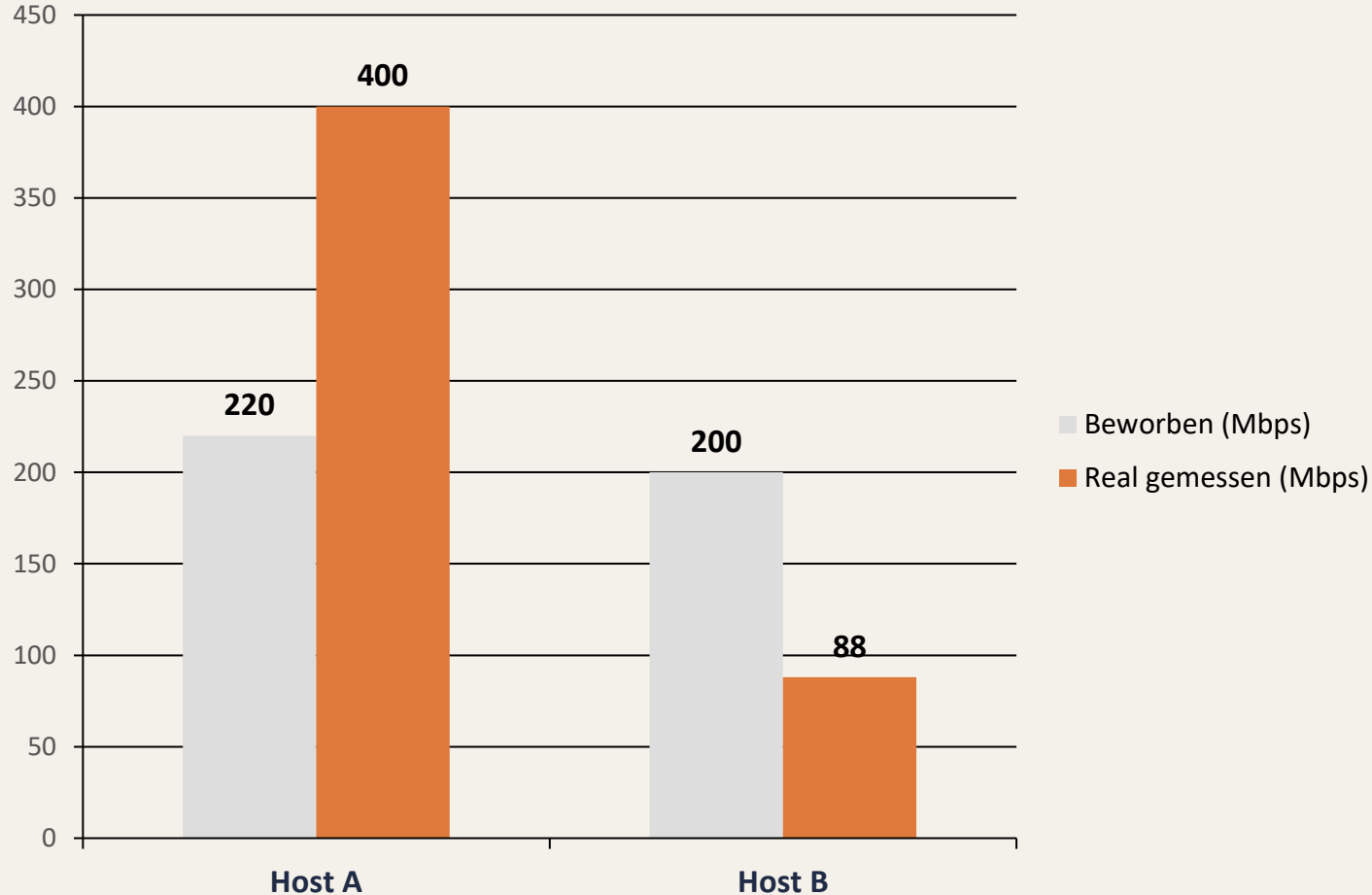
# Eigenen Key mitgeben
GPU_SUMMON_LLM_API_KEY=\
  meinkey \
python summon.py --solo-mode

# Bewusst offen
python summon.py \
  --solo-mode \
  --llm-api-key ''
# (nur in trusted networks!)
```

Hinweis: /v1/models ist upstream nicht authentifiziert — nur Inferenz-Endpunkte. Kein Bug, llama.cpp-Default.

Bandbreite: beworben vs. real

Eine Lektion, die im Workshop live zu sehen war



Was du da siehst:

Beworbene Mbps sind ein netter Marketing-Wert.

Die echte HuggingFace→Host-Geschwindigkeit kann deutlich darunter liegen — über 50% Shortfall ist keine Seltenheit.

Lösung: `--min-real-mbps`

1. Container hochfahren
2. SSH rein, Cache-File-Wachstum
15 s lang messen
3. Zu langsam? destroy + nächster Offer
4. Nach 5 Misses: Exit mit Tipp

Kosten pro fehlgeschlagenem Kandidaten: ~\$0.013 — billiger als 8 Min auf einem 80-Mbps-Host warten.

Kostenrahmen — was kommt da auf dich zu?

Zahlen aus April 2026, vast.ai · ähnliche Größenordnung bei anderen Anbietern

\$0.30 - 0.40

/h

RTX 5090 · Stundenpreis

\$0.40

≈ 1 h

interaktives Coding

\$3

≈ 8 h

Workshop-Tag

\$0.02 - 0.10

1×

Modell-Download (~21 GB)

Schutz gegen Ausreißer: Cost-Guards

```
python summon.py --solo-mode --max-price 0.45 --max-inet-cost 0.01
```

Über-Budget-Offer werden gar nicht erst betrachtet — harte Filter, kein 'upps zu teuer geworden'.

Lifecycle einer Session

Vier Befehle, die du wirklich brauchst

--list

Was läuft gerade?

Zeigt alle aktiven Instanzen mit ID, Modell, Preis, Uptime.

--ssh <id>

Reinschauen

Interaktive Shell auf der gemieteten Box.
tail -f /var/log/llama-server.log
nvidia-smi

--destroy <id>

Aufräumen

Container weg, Stundenzähler aus.
Pflicht — sonst läuft die Uhr.

--dry-run

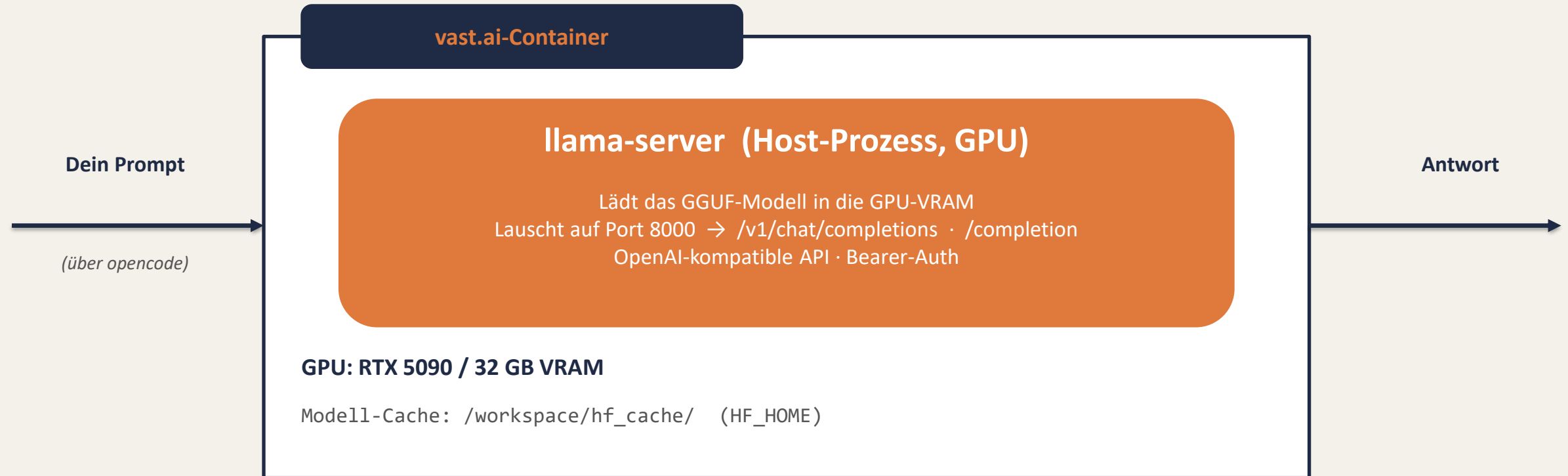
Erstmal ohne

Was würde gemietet, zu welchem Preis?
Erstellt nichts, druckt nur den Plan.

Faustregel: --list ist gratis. --destroy ist die Bremse. Vergiss sie nicht.

Was im Container läuft (einfache Variante)

Ein Prozess, ein Port, fertig



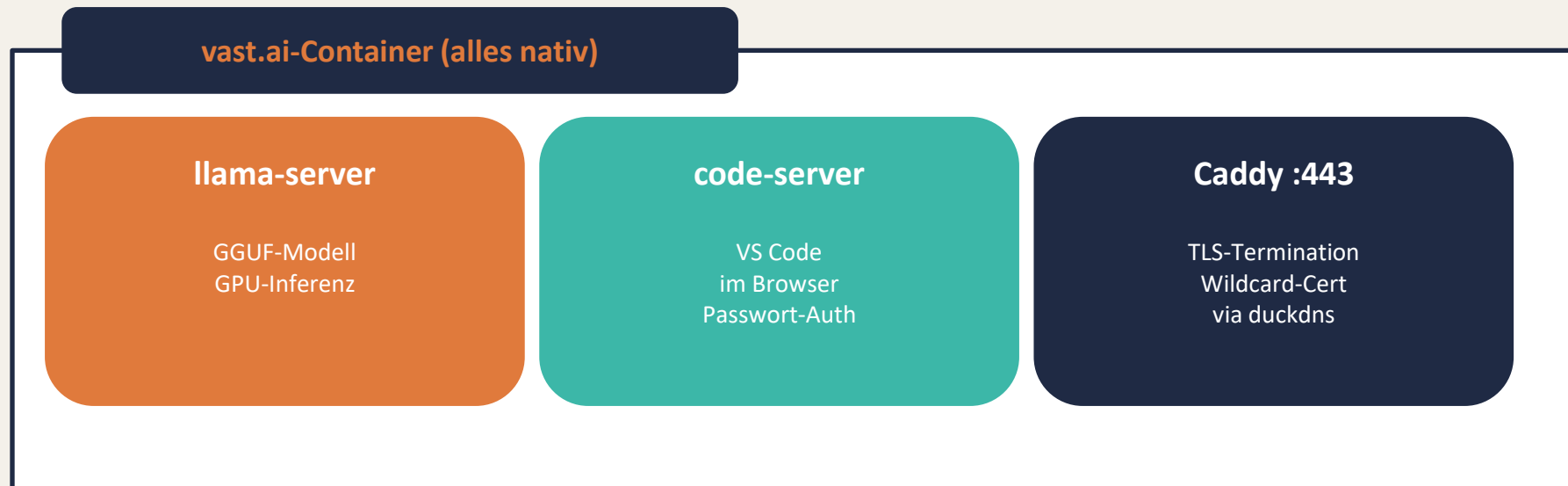
Einfach gehalten — keine Reverse-Proxy, kein Caddy, kein TLS-Zauber.

Vast.ai mappt einen zufälligen Host-Port auf 8000 — der Launcher druckt die finale URL.

Bonus — was nach dem Workshop dazukam

Optional: VS Code im Browser auf derselben Box

Nicht Teil der ursprünglichen Demo — als Erweiterung später hinzugefügt.



```
$ python summon.py --with-codeserver --code-domain mybox --solo-mode
```

Vorteil: kein Local-Setup nötig. Aufwand: kostenlose duckdns-Subdomain + Token.

Was du mitnimmst

Fünf Sätze, mit denen du am Montag was anfangen kannst

1

Lokale GPU ist nicht die einzige Option.

Eine gemietete für eine Stunde gibt es ab 30 ¢.

2

Ein Befehl reicht.

Suche, Mieten, Modell, Server, Config — vom Launcher erledigt.

3

Auth ist Default.

Bearer-Token wird generiert. Ohne ist absichtlich, nicht aus Versehen.

4

Beworbene Bandbreite ist Marketing.

Im Zweifel mit `--min-real-mbps` validieren — billiger als warten.

5

Vergiss `--destroy` nicht.

Sonst läuft die Stundenuhr weiter, auch wenn du den Laptop zuklappst.

Wie geht's weiter?

Repo, Docs, und was noch kommt

Repo & Docs

Hauptrepo (Source, README EN/DE)

`github.com/DG1001/gpu-summon`

Tiefgang — gesammelte Lessons

`LESSONS.md` (Bandbreite, `vast.ai`-Quirks, `llama.cpp`,
VRAM-Math, SSH-Recipes, Templates)

`vast.ai` Konto + API-Key

`cloud.vast.ai/manage-keys/`

opencode (lokaler Coding-Client)

`opencode.ai`

Was kommt noch?

- ➔ **TensorDock-Backend**
echte Public-IP, Port 443, `--privileged` erlaubt.
- ➔ **Mehr Provider**
`RunPod` / `Lambda` / `Salad` — gleicher CLI.
- ➔ **vLLM / SGLang**
alternative Inferenz-Engines neben `llama.cpp`.
- ➔ **Nativer CLI-Entry**
`gpu-summon` statt `python summon.py`.
- ✓ **`--with-codeserver`**
Browser-IDE inkl. TLS — schon da. ✓

Workshop-Material insgesamt: github.com/DG1001/jugs-agentic-workshop-2026

Danke für's Mitmachen!

Probier's einmal aus — eine Stunde,
weniger als ein Cappuccino, eigene Modelle.

github.com/DG1001/gpu-summon